Unit 3: Advanced Server Side Scripting

Complete Academic Master Edition

TU BCA 4th Semester Scripting Language (CACS252)

Prepared by: Gaurav Basyal **Total Hours:** 15 Hrs (OOP in PHP: 6 Hrs | AJAX: 3 Hrs | jQuery: 2 Hrs | Joomla: 3 Hrs | WordPress: 1 Hr)

Table of Contents

Module 3A: OOP Basics — Classes, Objects, Properties & Methods

1. Minute Rigorous Definition & Conceptual Core

Object-Oriented Programming (OOP) organises code around **objects** — self-contained units that bundle **data (properties)** and **behaviour (methods)** together — instead of a long sequence of procedural instructions.

- **Class:** A blueprint/template. It does not occupy memory for data until an object is created from it. Declared with the `class` keyword.
- **Object:** A concrete **instance** of a class, created using the `new` keyword. Each object has its own copy of non-static properties, but shares the same method code.
- **Property (Member Variable):** A variable declared inside a class that holds an object's state.
- **Method (Member Function):** A function declared inside a class that defines an object's behaviour. Methods can access the object's own properties using the special variable `$this`.
- **`$this`:** A pseudo-variable available only inside non-static methods. It always refers to the **calling object** — the specific instance the method was invoked on.
- **`->` (Object Operator):** Used to access properties and methods of an object instance (e.g., `$obj->name`, `$obj->getName()`).
- **Visibility Keywords:** `public`, `protected`, `private` control where a property/method can be accessed from (covered fully in Encapsulation, Module 3C).

2. Conceptual ASCII Diagram: Class → Object Relationship

```
CLASS "Student" (Blueprint — no memory for data yet)
+--------------------------------------+
| properties: $name, $dept, $score     |
| methods: setName(), getScore()       |
+--------------------------------------+
            |
new Student()   new Student()   new Student()
      |               |               |
      v               v               v
+-------------+ +-------------+ +-------------+
```

```
| OBJECT $s1   | | OBJECT $s2   | | OBJECT $s3   |
| name: Ram    | | name: Sita   | | name: Hari   |
| dept: BCA    | | dept: BIT    | | dept: BCA    |
| score: 85    | | score: 92    | | score: 70    |
+--------------+ +--------------+ +--------------+
```

Each object has its OWN copy of properties,
but the method CODE (setName, getScore) is shared.

3. Production-Ready Code Implementation

```php
<?php
// ============================================
// SECTION 1: Defining a Class
// ============================================

class Student {
    // Properties (member variables) hold each object's state
    public string $name;
    public string $dept;
    public float  $score;

    // Methods (member functions) define behaviour
    public function setName(string $name): void {
        $this->name = $name;   // $this refers to the CURRENT
        object
    }

    public function getScore(): float {
        return $this->score;
    }

    public function displayInfo(): string {
        return "Name: {$this->name} | Dept: {$this->dept} |
        Score: {$this->score}";
    }
}

// ============================================
// SECTION 2: Creating Objects (Instantiation)
// ============================================

$s1 = new Student();   // $s1 is an independent instance
$s1->name  = "Ram Prasad";
$s1->dept  = "BCA";
$s1->score = 85.0;
```

```php
$s2 = new Student();   // $s2 is a completely separate instance
$s2->setName("Sita Devi");   // Using a method instead of direct
        assignment
$s2->dept  = "BIT";
$s2->score = 92.0;

// ============================================
// SECTION 3: Accessing Properties & Methods
// ============================================

echo $s1->displayInfo() . "<br>";
echo $s2->displayInfo() . "<br>";
echo "Sita's score via method: " . $s2->getScore() . "<br>";

// Proof that objects are independent
$s1->score = 99.0;
echo "After changing s1 only -> s1: {$s1->score}, s2: {$s2->score}
        <br>";
?>
```

4. Comprehensive Analysis & Expected Output

Concept	Explanation
`class Student { }`	Defines the blueprint; no memory allocated yet
`new Student()`	Allocates memory and returns an object reference
`$this`	Inside a method, always points to the object the method was called on
`->`	Object operator; accesses properties/ methods
Typed properties (`public string $name`)	PHP 7.4+ enforces type declarations on class properties
Independence of objects	Changing `$s1` never affects `$s2` — they hold separate data

Expected Output:

```
Name: Ram Prasad | Dept: BCA | Score: 85
Name: Sita Devi | Dept: BIT | Score: 92
```

```
Sita's score via method: 92
After changing s1 only -> s1: 99, s2: 92
```

Module 3B: Constructors & Destructors

1. Minute Rigorous Definition & Conceptual Core

- **Constructor (`__construct()`):** A special "magic method" automatically invoked **the moment an object is created** with `new`. Its primary job is to initialise the object's properties, replacing repetitive manual assignment.
- **Constructor Parameters:** Constructors accept arguments just like normal functions, including default values (`$dept = "BCA"`) for optional parameters.
- **Constructor Promotion (PHP 8+):** Properties can be declared and assigned directly inside the constructor's parameter list, eliminating boilerplate.
- **Destructor (`__destruct()`):** A special magic method invoked automatically when an object is **no longer referenced**, or explicitly at the end of the script. Used for cleanup — closing file handles, DB connections, logging.
- **Object Lifecycle:** `new` (birth) → object used across the script → falls out of scope / `unset()` / script ends (death, `__destruct()` fires).

2. Conceptual ASCII Diagram: Object Lifecycle

```
new Student("Ram", "BCA")
          |
          v
  +------------------+
  |  __construct()     |   <-- Runs automatically, initialises
properties
  |  fires immediately  |
  +------------------+
          |
          v
  [ Object is alive — methods called, properties read/written ]
          |
  unset($obj)  OR  script ends  OR  variable goes out of scope
          |
          v
```

```
+-------------------+
|  __destruct()     |   <-- Runs automatically, cleanup logic
|  fires automatically|
+-------------------+
```

3. Production-Ready Code Implementation

```php
<?php
class Student {
    public string $name;
    public string $dept;
    public float  $score;

    // ============================================
    // Constructor: runs automatically on `new`
    // ============================================
    public function __construct(string $name, string $dept =
        "BCA", float $score = 0.0) {
        $this->name  = $name;
        $this->dept  = $dept;
        $this->score = $score;
        echo "Constructor called for $name<br>";
    }

    public function displayInfo(): string {
        return "Name: {$this->name} | Dept: {$this->dept} |
        Score: {$this->score}";
    }

    // ============================================
    // Destructor: runs automatically when object is destroyed
    // ============================================
    public function __destruct() {
        echo "Destructor called for {$this->name} — cleaning up
        resources.<br>";
    }
}

// ============================================
// PHP 8 Constructor Property Promotion (shorter syntax)
// ============================================
class Course {
    public function __construct(
        public string $title,
```

```php
        public int    $credits = 3   // Promoted property with
        default value
    ) {
        // No manual $this->title = $title needed — PHP does it
        automatically
    }
}

// ============================================
// Usage
// ============================================

$s1 = new Student("Ram Prasad", "BCA", 85.0);   // Constructor
        fires here
echo $s1->displayInfo() . "<br>";

$c1 = new Course("Scripting Language");         // Uses default
        credits = 3
echo "Course: {$c1->title} | Credits: {$c1->credits}<br>";

unset($s1);   // Explicitly destroys the object -> __destruct()
        fires now
echo "Script continues after unset()...<br>";
// If not unset manually, __destruct() fires automatically at
        script end.
?>
```

4. Comprehensive Analysis & Expected Output

Magic Method	Trigger	Common Use
`__construct()`	Object creation (`new`)	Initialise properties, open connections
`__destruct()`	`unset()`, scope end, or script end	Close files/DB connections, logging
Constructor promotion	PHP 8+ shorthand	Reduces boilerplate for simple data classes
Default parameters	`$dept = "BCA"`	Allows omitting arguments at call site

Expected Output:

```
Constructor called for Ram Prasad
Name: Ram Prasad | Dept: BCA | Score: 85
Course: Scripting Language | Credits: 3
```

```
Destructor called for Ram Prasad — cleaning up resources.
Script continues after unset()...
```

Module 3C: Encapsulation & Method Overriding

1. Minute Rigorous Definition & Conceptual Core

Encapsulation is the bundling of data and the methods that operate on it, while **restricting direct outside access** to some of an object's internal details — protecting data integrity.

Visibility Modifiers:

Modifier	Accessible From
`public`	Anywhere — inside the class, subclasses, and outside code
`protected`	Inside the class and any subclass (child class), **not** from outside
`private`	Only inside the exact class it is declared in — not even subclasses

- **Getters & Setters:** Public methods (`getScore()`, `setScore()`) that provide **controlled** access to `private`/`protected` properties, allowing validation logic before a value is accepted.
- **Why encapsulate?** Without it, external code could set `$score = -50` or `$score = "abc"` directly. A setter method can validate and reject invalid data.
- **Method Overriding:** When a **child class** redefines a method that already exists in its **parent class**, using the exact same method signature. The child's version replaces the parent's version when called on a child object. (Requires inheritance — see Module 3D — but is introduced here since it works hand-in-hand with encapsulated design.)
- **`parent::`:** Inside an overriding method, `parent::methodName()` explicitly calls the parent class's original version, useful for extending rather than fully replacing behaviour.

2. Conceptual ASCII Diagram: Encapsulation Access Control

```
                class Student
        +---------------------------------+
        | private   $score      <-- Hidden, only class itself can
touch |
        | protected $dept       <-- Class + subclasses
only             |
        | public    $name       <-- Anyone can
touch                    |

|                                                                  |
        | public function setScore($val)
{                              |
        |     if ($val >= 0 && $val <= 100) {  //
VALIDATION             |
        |         $this->score =
$val;                                  |

|     }                                                            |

| }                                                                 |
        +---------------------------------+
                        ^
                        |  Only route external code can use
                        |  to change $score safely
              [ Outside Code ]  $s->setScore(150);  --> Rejected
by validation
                                  $s->score = 150;    --> FATAL
ERROR (private)
```

3. Production-Ready Code Implementation

```php
<?php
// ============================================
// SECTION 1: Encapsulation with Private/Protected
// ============================================

class Student {
    public    string $name;
    protected string $dept;      // Accessible in this class +
        child classes
```

```php
    private   float  $score;     // Accessible ONLY inside this
        exact class

    public function __construct(string $name, string $dept) {
        $this->name = $name;
        $this->dept = $dept;
        $this->score = 0.0;
    }

    // Setter: validates before allowing a change to the private
        property
    public function setScore(float $value): void {
        if ($value >= 0 && $value <= 100) {
            $this->score = $value;
        } else {
            echo "Invalid score rejected: $value<br>";
        }
    }

    // Getter: controlled READ access to the private property
    public function getScore(): float {
        return $this->score;
    }

    public function displayInfo(): string {
        return "Name: {$this->name} | Dept: {$this->dept} |
        Score: {$this->score}";
    }
}

$s1 = new Student("Ram Prasad", "BCA");
$s1->setScore(85.0);      // Accepted
$s1->setScore(150.0);     // Rejected by validation logic
echo $s1->displayInfo() . "<br>";
// echo $s1->score;       // FATAL ERROR if uncommented — private
    property

// ================================================
// SECTION 2: Method Overriding (with Inheritance)
// ================================================

class GraduateStudent extends Student {
    private string $thesisTitle;

    public function __construct(string $name, string $dept,
        string $thesisTitle) {
```

```
        parent::__construct($name, $dept);   // Reuse parent
        constructor logic
        $this->thesisTitle = $thesisTitle;
    }

    // OVERRIDING displayInfo(): same name/signature as parent,
        new behaviour
    public function displayInfo(): string {
        $base = parent::displayInfo();       // Call the ORIGINAL
        parent version
        return $base . " | Thesis: {$this->thesisTitle}";
    }
}

$g1 = new GraduateStudent("Sita Devi", "BIT", "AI in Healthcare");
$g1->setScore(92.0);
echo $g1->displayInfo() . "<br>";   // Uses the OVERRIDDEN version
?>
```

4. Comprehensive Analysis & Expected Output

Feature	Behaviour
`private $score`	Fatal error if accessed as `$obj->score` from outside the class
`protected $dept`	Not accessible outside, but `GraduateStudent` (child) can use it internally
Setter validation	`setScore(150)` silently rejected — data integrity preserved
Method Overriding	`GraduateStudent::displayInfo()` replaces `Student::displayInfo()` when called on a `GraduateStudent` object
`parent::displayInfo()`	Reuses parent logic instead of duplicating it — extends rather than replaces

Expected Output:

```
Invalid score rejected: 150
Name: Ram Prasad | Dept: BCA | Score: 85
Name: Sita Devi | Dept: BIT | Score: 92 | Thesis: AI in Healthcare
```

Module 3D: Inheritance & Polymorphism

1. Minute Rigorous Definition & Conceptual Core

- **Inheritance:** A mechanism where a **child class** (subclass) acquires the properties and methods of a **parent class** (superclass) using the `extends` keyword, promoting code reuse and hierarchical modelling ("is-a" relationships, e.g., a `GraduateStudent` **is a** `Student`).
- **Single Inheritance:** PHP classes can extend only **one** parent class directly (unlike interfaces, which support multiple implementation).
- **Abstract Classes:** Declared with `abstract class`. Cannot be instantiated directly (`new AbstractClass()` throws an error) — they exist only to be extended. May contain **abstract methods** (`abstract public function calculate();`) which have **no body** and **must** be implemented by every concrete child class.
- **Interfaces:** Declared with `interface`. Define a **contract** — a list of method signatures with no implementation at all. A class uses `implements` to guarantee it provides those methods. Unlike abstract classes, a class **can implement multiple interfaces**.
- **Polymorphism ("many forms"):** The ability to treat objects of **different classes** through a **common parent type or interface**, while each object responds to the same method call in its own specific way. This is the payoff of inheritance + method overriding.

2. Conceptual ASCII Diagram: Inheritance Hierarchy & Polymorphism

```
                abstract class Shape
              +--------------------------+
              | abstract function area()   |
              +--------------------------+
                  ^              ^
                  |              |
     class Circle          class Rectangle
  +----------------+   +--------------------+
  | area() {         |   | area() {            |
  |   pi * r * r      |   |   width * height    |
  | }                |   | }                   |
  +----------------+   +--------------------+

POLYMORPHISM IN ACTION:
$shapes = [ new Circle(5), new Rectangle(4, 6) ];
```

```
  foreach ($shapes as $shape) {
      echo $shape->area();   // Same method call, DIFFERENT logic
per object
  }
```

3. Production-Ready Code Implementation

```php
<?php
// ============================================
// SECTION 1: Abstract Class (Partial Blueprint)
// ============================================

abstract class Shape {
    protected string $name;

    public function __construct(string $name) {
        $this->name = $name;
    }

    // Abstract method: NO body — every child MUST implement this
    abstract public function area(): float;

    // Regular concrete method: shared by all children
        automatically
    public function describe(): string {
        // Calls area() polymorphically — resolves to whichever
        child implemented it
        return "{$this->name} has area: " . round($this->area(),
        2);
    }
}

// ============================================
// SECTION 2: Concrete Child Classes (Inheritance)
// ============================================

class Circle extends Shape {
    private float $radius;

    public function __construct(float $radius) {
        parent::__construct("Circle");
        $this->radius = $radius;
    }
```

```php
    public function area(): float {           // Required
        implementation
        return pi() * $this->radius ** 2;
    }
}

class Rectangle extends Shape {
    private float $width;
    private float $height;

    public function __construct(float $width, float $height) {
        parent::__construct("Rectangle");
        $this->width  = $width;
        $this->height = $height;
    }

    public function area(): float {           // Different
        implementation
        return $this->width * $this->height;
    }
}

// ================================================
// SECTION 3: Interface (Pure Contract, Multiple Implementation)
// ================================================

interface Printable {
    public function toJson(): string;
}

// A class can extend ONE class but implement MULTIPLE interfaces
class Square extends Shape implements Printable {
    private float $side;

    public function __construct(float $side) {
        parent::__construct("Square");
        $this->side = $side;
    }

    public function area(): float {
        return $this->side ** 2;
    }

    public function toJson(): string {         // Fulfils the
        interface contract
```

```
        return json_encode(["name" => $this->name, "area" =>
        $this->area()]);
    }
}

// ============================================
// SECTION 4: Polymorphism in Action
// ============================================

$shapes = [
    new Circle(5),
    new Rectangle(4, 6),
    new Square(3),
];

foreach ($shapes as $shape) {
    // Every object is treated as a "Shape" — but area() runs its
        OWN version
    echo $shape->describe() . "<br>";
}

$sq = new Square(4);
echo "JSON: " . $sq->toJson() . "<br>";

// echo (new Shape("test"))->area();  // FATAL ERROR — cannot
        instantiate abstract class
?>
```

4. Comprehensive Analysis & Expected Output

Concept	Key Rule
`abstract class`	Cannot be instantiated; forces children to implement abstract methods
`extends`	Single inheritance only — one direct parent
`interface`	Pure contract, no implementation; a class can implement several
`implements`	Guarantees the class provides every method the interface declares
Polymorphism	Same method call (`area()`), different result depending on the actual object type
`describe()`	Defined once in the parent, works correctly for **every** child without modification

Expected Output:

```
Circle has area: 78.54
Rectangle has area: 24
Square has area: 9
JSON: {"name":"Square","area":16}
```

Module 3E: Static Members

1. Minute Rigorous Definition & Conceptual Core

- **Static Property:** A property declared with the `static` keyword that belongs to the **class itself**, not to any individual object. All instances share **exactly one copy** of a static property.
- **Static Method:** A method declared `static` that can be called **without creating an object** (`ClassName::method()`). Inside a static method, `$this` is **not available** (there is no calling object), but `self::` or `static::` can access other static members.
- **`self::` vs `static::`:** `self::` refers to the class where the code is physically written (early binding). `static::` refers to the class that was actually called at runtime (late static binding) — relevant in inheritance scenarios.
- **`::` (Scope Resolution Operator):** Used to access static properties, static methods, class constants, and parent methods, without needing an object instance.
- **Common Use Case:** Tracking data shared across **all** instances — e.g., a running count of how many objects have been created.

2. Conceptual ASCII Diagram: Static vs Instance Memory

```
                    class Student
          +-----------------------------------+
          | static $totalStudents = 0  <-- ONE copy, shared by ALL
objects |
          +-----------------------------------+
                  |          |          |
        +--------+  +------+--+  +---+------+
        | $s1      |  | $s2      |  | $s3      |
        | name:Ram |  | name:Sita|  | name:Hari|
        +----------+  +----------+  +----------+
        Each object has its OWN "name",
```

```
       but they all read/write the SAME $totalStudents.

       Access:  Student::$totalStudents   (via class, no object
needed)
       NOT:     $s1->totalStudents        (wrong syntax for
static)
```

3. Production-Ready Code Implementation

```php
<?php
class Student {
    public string $name;

    // Static property: shared across ALL Student objects
    public static int $totalStudents = 0;

    // Class constant: fixed value, cannot change, accessed via ::
    const MAX_SCORE = 100;

    public function __construct(string $name) {
        $this->name = $name;
        self::$totalStudents+
        +;   // Increments the SHARED counter on every
        construction
    }

    // Static method: called on the CLASS, not an object
    public static function getTotalStudents(): int {
        return self::$totalStudents;
    }

    public static function passMarkInfo(): string {
        return "Maximum possible score is " . self::MAX_SCORE;
    }
}

// ============================================
// Usage
// ============================================

echo "Before creating any student: " .
        Student::getTotalStudents() . "<br>";

$s1 = new Student("Ram Prasad");
$s2 = new Student("Sita Devi");
```

```php
$s3 = new Student("Hari Bahadur");

// Called via CLASS NAME, not via any object
echo "Total students created: " . Student::getTotalStudents() .
        "<br>";
echo "Also accessible directly: " . Student::$totalStudents .
        "<br>";
echo Student::passMarkInfo() . "<br>";
echo "Max score constant: " . Student::MAX_SCORE . "<br>";
?>
```

4. Comprehensive Analysis & Expected Output

Feature	Syntax	Notes
Static property	`public static $x`	One copy shared by every instance
Static method	`public static function f()`	Callable without `new`; no `$this` inside
Scope resolution `::`	`ClassName::$prop`, `ClassName::method()`	Used for static members and constants
`self::`	Refers to the class the code is written in	Early binding
Class constant	`const NAME = value;`	Immutable, accessed with `::`, no `$` prefix

Expected Output:

```
Before creating any student: 0
Total students created: 3
Also accessible directly: 3
Maximum possible score is 100
Max score constant: 100
```

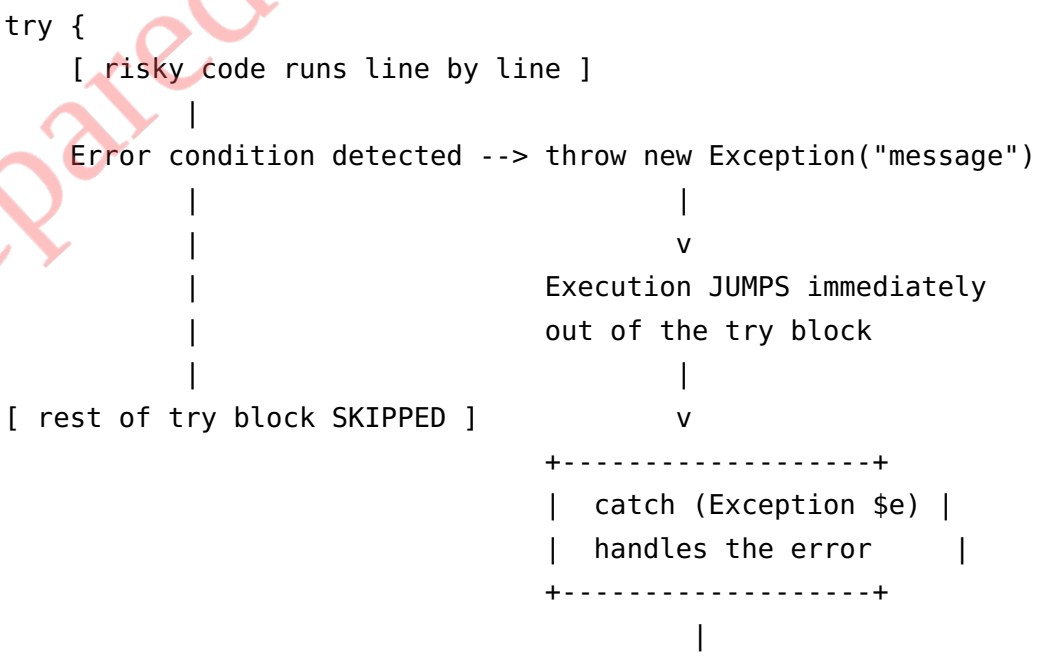
Module 3F: Exception Handling

1. Minute Rigorous Definition & Conceptual Core

Exception Handling provides a structured way to detect and respond to runtime errors, without letting the script crash abruptly or silently produce wrong results.

- **`try` block:** Wraps code that **might** throw an error.
- **`throw`:** Manually raises an exception object when an abnormal condition is detected (e.g., division by zero, invalid input).
- **`catch` block:** Intercepts a thrown exception of a matching type and handles it gracefully (logging, user-friendly message, fallback logic). Multiple `catch` blocks can handle different exception types for the same `try`.
- **`finally` block:** Executes **always**, whether an exception was thrown or not — ideal for guaranteed cleanup (closing a file/DB connection).
- **`Exception` class:** PHP's built-in base exception class. Custom exceptions are created by **extending** it (`class InvalidScoreException extends Exception {}`), allowing very specific `catch` targeting.
- **Exception object methods:** `getMessage()`, `getCode()`, `getLine()`, `getFile()` provide diagnostic details about what went wrong and where.

2. Conceptual ASCII Diagram: Exception Flow Control



```
                                        v
                                  +------------------+
                                  |  finally { }     |  <--
ALWAYS runs
                                  +------------------+
```

3. Production-Ready Code Implementation

```php
<?php
// ============================================
// SECTION 1: Custom Exception Class
// ============================================

class InvalidScoreException extends Exception {
    // Inherits getMessage(), getCode(), getLine() automatically
}

class DatabaseConnectionException extends Exception {}

// ============================================
// SECTION 2: A Function That Throws Exceptions
// ============================================

function recordScore(float $score): string {
    if ($score < 0 || $score > 100) {
        // throw halts normal execution and hands control to the
        nearest matching catch
        throw new InvalidScoreException("Score $score is out of
        the valid range (0-100).");
    }
    return "Score $score recorded successfully.";
}

// ============================================
// SECTION 3: try / catch / finally
// ============================================

$testScores = [85, 150, 42, -10];

foreach ($testScores as $score) {
    try {
        echo recordScore($score) . "<br>";
    } catch (InvalidScoreException $e) {
        // Catches ONLY InvalidScoreException (or its subclasses)
```

```
        echo "Validation Error: " . $e->getMessage() . " (Line:
        " . $e->getLine() . ")<br>";
    } catch (Exception $e) {
        // Fallback: catches any OTHER exception type not already
        handled above
        echo "General Error: " . $e->getMessage() . "<br>";
    } finally {
        // Runs after EVERY iteration, success or failure
        echo "-- processed score: $score --<br>";
    }
}

// ============================================
// SECTION 4: Multiple catch Types & Real-World DB Example
// ============================================

function connectToDatabase(bool $simulateFailure): string {
    if ($simulateFailure) {
        throw new DatabaseConnectionException("Could not reach
        MySQL server on port 3306.");
    }
    return "Connected successfully.";
}

try {
    echo connectToDatabase(true);
} catch (DatabaseConnectionException $e) {
    echo "DB Error: " . $e->getMessage() . "<br>";
} catch (InvalidScoreException $e) {
    echo "Score Error: " . $e->getMessage() . "<br>";
} finally {
    echo "Connection attempt finished — releasing resources.<br>";
}
?>
```

4. Comprehensive Analysis & Expected Output

Keyword	Role
`try`	Marks code that might fail
`throw`	Manually raises an exception object
`catch (Type $e)`	Intercepts exceptions of a specific type (checked top to bottom, first match wins)
`finally`	Always executes, regardless of whether an exception occurred

Keyword	Role
`extends Exception`	Creates custom, specific exception types for precise error handling
`getMessage()` / `getLine()`	Built-in diagnostic methods inherited from the base `Exception` class

Expected Output:

```
Score 85 recorded successfully.
-- processed score: 85 --
Validation Error: Score 150 is out of the valid range (0-100).
(Line: 17)
-- processed score: 150 --
Score 42 recorded successfully.
-- processed score: 42 --
Validation Error: Score -10 is out of the valid range (0-100).
(Line: 17)
-- processed score: -10 --
DB Error: Could not reach MySQL server on port 3306.
Connection attempt finished — releasing resources.
```

Module 3G: AJAX Fundamentals — Using PHP

1. Minute Rigorous Definition & Conceptual Core

AJAX (Asynchronous JavaScript and XML) is a technique — not a language — that lets a web page communicate with a server **in the background**, updating parts of the page **without a full reload**. Despite the name, modern AJAX almost always exchanges **JSON**, not XML.

- **Synchronous vs Asynchronous:** A traditional form submission is synchronous — the browser waits, blanks the page, and reloads it entirely. AJAX is asynchronous — JavaScript sends a request, keeps the page fully interactive, and updates only a specific DOM element once the response arrives.
- **`XMLHttpRequest` (XHR):** The original browser object for making AJAX calls. Still supported everywhere.
- **`fetch()` API:** The modern, Promise-based replacement for `XMLHttpRequest`. Cleaner syntax, native support for JSON.

- **Server-Side Role of PHP:** The PHP endpoint receives the request like any normal request (reads `$_GET`/`$_POST`), but instead of returning a full HTML page, it returns a **small fragment** — plain text, JSON, or an HTML snippet — meant to be inserted into the existing page.
- **`header('Content-Type: application/json');`:** Tells the browser to interpret the PHP response as JSON, which `fetch()`/`XHR` can then parse automatically.

2. Conceptual ASCII Diagram: AJAX Request Lifecycle

```
TRADITIONAL FORM SUBMIT:                    AJAX REQUEST:
Browser --(full page POST)--> Server        Browser --(background
fetch)--> Server
Server --(entire new HTML)--> Browser       Server --(JSON
fragment)------> Browser
   |                                            |
[ Full page reloads, flickers ]         [ JavaScript updates ONE
<div>,
                                                rest of page stays
untouched ]

AJAX SEQUENCE:
[User Action: e.g. types in a box]
        |
        v
[ JavaScript: fetch('check.php?name=Ram') ]
        |
        v
[ PHP: check.php runs, echoes JSON: {"status":"available"} ]
        |
        v
[ JavaScript: .then(res => res.json()) -> updates <div
id="result"> ]
```

3. Production-Ready Code Implementation

`check_username.php` (server-side endpoint):

```php
<?php
// ============================================
// AJAX Endpoint: Returns JSON, not a full HTML page
// ============================================
```

```php
header('Content-Type: application/json');   // Tell the browser
        this is JSON

$takenUsernames = ["ram_dev", "admin", "sita92"];

$username = htmlspecialchars(trim($_GET['username'] ?? ''));

$response = [];

if (empty($username)) {
    $response = ["status" => "error", "message" => "Username
        cannot be empty."];
} elseif (in_array($username, $takenUsernames)) {
    $response = ["status" => "taken", "message" =>
        "'$username' is already taken."];
} else {
    $response = ["status" => "available", "message" =>
        "'$username' is available!"];
}

echo json_encode($response);   // Encodes PHP array as a JSON
        string
exit;
?>
```

index.html (client-side, using the modern `fetch()` API):

```html
<!DOCTYPE html>
<html>
<head>
    <title>AJAX Username Checker</title>
</head>
<body>
    <input type="text" id="username" placeholder="Enter username">
    <button onclick="checkUsername()">Check Availability</button>
    <div id="result"></div>

    <script>
    function checkUsername() {
        const username =
        document.getElementById('username').value;

        // fetch() sends the request WITHOUT reloading the page
        fetch('check_username.php?username=' +
        encodeURIComponent(username))
            .then(response => response.json())     // Parse JSON
        response body
            .then(data => {
```

```
                const resultDiv =
            document.getElementById('result');
                resultDiv.textContent = data.message;
                resultDiv.style.color = (data.status ===
            'available') ? 'green' : 'red';
            })
            .catch(error => console.error('AJAX Error:', error));
    }
    </script>
</body>
</html>
```

Equivalent using the classic `XMLHttpRequest` object:

```
function checkUsernameXHR() {
    const username = document.getElementById('username').value;
    const xhr = new XMLHttpRequest();

    xhr.open('GET', 'check_username.php?username=' +
        encodeURIComponent(username), true); // true = async

    xhr.onreadystatechange = function () {
        // readyState 4 = DONE, status 200 = OK
        if (xhr.readyState === 4 && xhr.status === 200) {
            const data = JSON.parse(xhr.responseText);
            document.getElementById('result').textContent =
        data.message;
        }
    };

    xhr.send();   // Fires the request; script continues without
        waiting
}
```

4. Comprehensive Analysis & Expected Output

Component	Role
`fetch()` / `XMLHttpRequest`	Client-side JavaScript objects that send background HTTP requests
`check_username.php`	Server endpoint; behaves like any PHP script but returns JSON, not HTML
`header('Content-Type: application/json')`	Correct MIME type so the browser/JS parses the response properly
`json_encode()`	Converts a PHP array into a JSON string

Component	Role
`.then(response => response.json())`	Parses the raw response body into a usable JavaScript object
Page behaviour	Only `#result` updates; the rest of the page never reloads

Expected Behaviour (typing "admin" then clicking Check):

```
Server response: {"status":"taken","message":"'admin' is already taken."}
Page displays (in red, no reload): 'admin' is already taken.
```

Module 3H: AJAX with PHP + MySQL

1. Minute Rigorous Definition & Conceptual Core

Combining AJAX with a MySQL backend allows a page to **fetch, insert, or search live database data** in the background — the foundation of features like live search, dynamic dropdowns (e.g., select a district, populate its municipalities), and instant form validation.

- **Pattern:** JavaScript sends a request → PHP receives it, runs a MySQL query with `mysqli` → PHP encodes the result set as JSON with `json_encode()` → JavaScript parses the JSON and updates the DOM.
- **Prepared Statements:** Because AJAX endpoints directly accept user-supplied input (search terms, IDs), **prepared statements** (`mysqli_prepare()` / bound parameters) are essential to prevent **SQL Injection** — never concatenate raw `$_GET`/`$_POST` values into a SQL string.
- **`mysqli_fetch_all(MYSQLI_ASSOC)`:** Fetches an **entire result set** as an array of associative arrays in one call — convenient for feeding directly into `json_encode()`.

2. Conceptual ASCII Diagram: AJAX + MySQL Live Search Flow

```
[Browser: user types "Ram" in a search box]
            |
            v
   fetch('search_students.php?q=Ram')
```

```
          |
          v
  +-------------------------------------------------+
  |  search_students.php                             |
  |    1. Receives $_GET['q'] = "Ram"                  |
  |    2. Prepared statement: WHERE name LIKE ?        |
  |    3. mysqli_stmt_execute()                        |
  |    4. Fetch matching rows from MySQL               |
  |    5. json_encode(rows) --> outputs JSON array     |
  +-------------------------------------------------+
          |
          v
  [ JavaScript receives JSON, loops through it,
    dynamically builds <li> list items in the page ]
```

3. Production-Ready Code Implementation

`search_students.php` (AJAX endpoint with a secure, parameterised MySQL query):

```php
<?php
header('Content-Type: application/json');

$conn = mysqli_connect("localhost", "root", "", "tu_lab_db");
if (!$conn) {
    echo json_encode(["error" => "Database connection failed."]);
    exit;
}

$searchTerm = trim($_GET['q'] ?? '');

if (strlen($searchTerm) < 1) {
    echo json_encode([]);   // Empty search -> empty result set
    exit;
}

// ============================================
// PREPARED STATEMENT: prevents SQL Injection
// ============================================
$sql = "SELECT id, name, dept, score FROM students WHERE name
        LIKE ? LIMIT 10";
$stmt = mysqli_prepare($conn, $sql);

$likeTerm = "%" . $searchTerm . "%";
```

```php
mysqli_stmt_bind_param($stmt, "s", $likeTerm);   // "s" = string
        parameter type
mysqli_stmt_execute($stmt);

$result  = mysqli_stmt_get_result($stmt);
$students = mysqli_fetch_all($result, MYSQLI_ASSOC);   // Entire
        result -> array

echo json_encode($students);   // Output as a JSON array of
        objects

mysqli_stmt_close($stmt);
mysqli_close($conn);
?>
```

live_search.html (client-side live search UI):

```html
<!DOCTYPE html>
<html>
<head><title>Live Student Search</title></head>
<body>
    <input type="text" id="searchBox" placeholder="Search student
        name..." onkeyup="liveSearch()">
    <ul id="resultsList"></ul>

    <script>
    function liveSearch() {
        const query = document.getElementById('searchBox').value;

        fetch('search_students.php?q=' +
        encodeURIComponent(query))
            .then(response => response.json())
            .then(students => {
                const list =
        document.getElementById('resultsList');
                list.innerHTML = '';   // Clear previous results

                if (students.length === 0) {
                    list.innerHTML = '<li>No matches found.</li>';
                    return;
                }

                // Dynamically build the list from JSON data
                students.forEach(s => {
                    const li = document.createElement('li');
                    li.textContent = `${s.name} (${s.dept}) —
        Score: ${s.score}`;
```

```
                    list.appendChild(li);
                });
            })
            .catch(err => console.error('Search failed:', err));
    }
    </script>
</body>
</html>
```

insert_student_ajax.php (AJAX INSERT example, receiving JSON POST data):

```
<?php
header('Content-Type: application/json');
$conn = mysqli_connect("localhost", "root", "", "tu_lab_db");

// AJAX POST bodies sent as JSON must be read from php://input,
        NOT $_POST
$data = json_decode(file_get_contents("php://input"), true);

$name  = mysqli_real_escape_string($conn, $data['name']  ?? '');
$dept  = mysqli_real_escape_string($conn, $data['dept']  ?? '');
$score = floatval($data['score'] ?? 0);

$stmt = mysqli_prepare($conn, "INSERT INTO students (name, dept,
        score) VALUES (?, ?, ?)");
mysqli_stmt_bind_param($stmt, "ssd", $name, $dept, $score);

if (mysqli_stmt_execute($stmt)) {
    echo json_encode(["status" => "success", "new_id" =>
        mysqli_insert_id($conn)]);
} else {
    echo json_encode(["status" => "error", "message" =>
        mysqli_error($conn)]);
}

mysqli_stmt_close($stmt);
mysqli_close($conn);
?>
```

4. Comprehensive Analysis & Expected Output

Function	Purpose
`mysqli_prepare()`	

Function	Purpose
	Compiles a parameterised SQL template — placeholders (`?`) hold user input safely
`mysqli_stmt_bind_param(stmt, "s", $var)`	Binds a value to a placeholder; type letters: `s`=string, `i`=int, `d`=double
`mysqli_fetch_all($result, MYSQLI_ASSOC)`	Converts the entire result set into a PHP array in one line
`php://input`	Raw request body stream — required to read JSON sent via `fetch()` POST
Live search pattern	Runs on every keystroke (`onkeyup`), giving instant feedback without reloading

Expected Behaviour (typing "Ram" in the search box):

```
GET request: search_students.php?q=Ram
JSON response: [{"id":"1","name":"Ram
Prasad","dept":"BCA","score":"85"}]
Page dynamically renders: "Ram Prasad (BCA) — Score: 85"
```

Module 3I: jQuery — Playing with Elements, Hiding/Unhiding Images & jQuery UI

1. Minute Rigorous Definition & Conceptual Core

jQuery is a lightweight JavaScript library that dramatically simplifies DOM manipulation, event handling, animation, and AJAX calls, using a concise, chainable syntax.

- **The `$` Function:** `$(selector)` wraps matched DOM elements in a jQuery object, exposing dozens of convenience methods. `$("#id")`, `$(".class")`, `$("tag")` mirror CSS selector syntax.
- **Document Ready:** `$(document).ready(function(){ ... })` (or shorthand `$(function(){ ... })`) ensures code only runs after the full HTML DOM has loaded, preventing "element not found" errors.
- **Method Chaining:** jQuery methods return the jQuery object itself, allowing calls to be chained: `$("#box").fadeOut().delay(500).fadeIn();`

- **`.hide()` / `.show()` / `.toggle()`:** Instantly (or with animation duration/speed) change an element's CSS `display` property.
- **`.fadeIn()` / `.fadeOut()` / `.fadeToggle()`:** Animate opacity over a duration for a smoother visual transition than instant hide/show.
- **`.slideUp()` / `.slideDown()` / `.slideToggle()`:** Animate an element's height, creating a collapsing/expanding effect.
- **jQuery UI:** A companion library built **on top of** jQuery, providing ready-made interactive widgets (datepicker, dialog, accordion, tabs, slider) and pre-built animation effects, saving developers from writing complex interaction logic from scratch.

2. Conceptual ASCII Diagram: jQuery Selection & Manipulation Flow

```
   HTML DOM Tree                        jQuery Wraps & Manipulates
   +-------------+                      +--------------------------+
   | <img id=     |   $("#photo")        | jQuery Object
|
   |   "photo">   | ----------------> | .hide()  -> display:none
|
   +-------------+                      | .fadeIn()-> opacity 0->1
|
                                        | .toggle()-> flip state
|
                                        +--------------------------
+


   EVENT BINDING:
   $("#toggleBtn").click(function() {
       $("#photo").toggle();     <-- Runs EVERY time the button is
clicked
   });
```

3. Production-Ready Code Implementation

```
<!DOCTYPE html>
<html>
<head>
    <title>jQuery Elements & Effects</title>
    <!-- jQuery library from CDN -->
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></
        script>
    <!-- jQuery UI CSS & JS -->
```

```
    <link rel="stylesheet" href="https://code.jquery.com/ui/
        1.13.2/themes/base/jquery-ui.css">
    <script src="https://code.jquery.com/ui/1.13.2/jquery-
        ui.min.js"></script>
</head>
<body>

    <h3>Section 1: Hiding & Unhiding an Image</h3>
    <img id="photo" src="student.jpg" width="200" alt="Student
        photo">
    <br>
    <button id="hideBtn">Hide Image</button>
    <button id="showBtn">Show Image</button>
    <button id="toggleBtn">Toggle Image</button>
    <button id="fadeBtn">Fade Toggle</button>

    <h3>Section 2: Playing with Elements</h3>
    <ul id="studentList">
        <li>Ram Prasad</li>
        <li>Sita Devi</li>
    </ul>
    <button id="addBtn">Add Student</button>
    <button id="removeBtn">Remove Last</button>
    <button id="colorBtn">Highlight List</button>

    <h3>Section 3: jQuery UI Widgets</h3>
    <p>Pick a date: <input type="text" id="datepicker"></p>
    <div id="dialogBox" title="Notice">This is a jQuery UI dialog
        box.</div>
    <button id="openDialog">Open Dialog</button>

    <script>
    // ============================================
    // Document Ready: code runs only after DOM loads
    // ============================================
    $(document).ready(function () {

        // ---------- SECTION 1: Hide / Show / Toggle / Fade
        ----------
        $("#hideBtn").click(function () {
            $("#photo").hide(500);          // Animates over 500ms
        });

        $("#showBtn").click(function () {
            $("#photo").show(500);
        });
```

```
        $("#toggleBtn").click(function () {
            $("#photo").toggle(500);        // Switches between
        hide/show each click
        });

        $("#fadeBtn").click(function () {
            $("#photo").fadeToggle(800);    // Smooth opacity
        transition
        });

        // ---------- SECTION 2: Playing with Elements ----------
        $("#addBtn").click(function () {
            // .append() inserts new content as the LAST child
            $("#studentList").append("<li>New Student " + ($
        ("#studentList li").length + 1) + "</li>");
        });

        $("#removeBtn").click(function () {
            $("#studentList li:last").remove();  // Removes the
        last <li>
        });

        $("#colorBtn").click(function () {
            // .css() reads or sets CSS properties directly
            $("#studentList li").css("background-color",
        "yellow").css("padding", "5px");
        });

        // ---------- SECTION 3: jQuery UI Widgets ----------
        $("#datepicker").datepicker();     // Turns a plain input
        into a calendar widget

        $("#dialogBox").dialog({
            autoOpen: false,                // Don't show until
        triggered
            modal: true
        });

        $("#openDialog").click(function () {
            $("#dialogBox").dialog("open");
        });
    });
    </script>
</body>
</html>
```

4. Comprehensive Analysis & Expected Output

Method	Effect
`.hide(speed)` / `.show(speed)`	Instantly or gradually sets `display: none` / restores original display
`.toggle(speed)`	Switches between hidden and visible on each call
`.fadeIn()` / `.fadeOut()` / `.fadeToggle()`	Animates the `opacity` property
`.append()`	Inserts HTML as the last child of the selected element
`.remove()`	Deletes the selected element(s) entirely from the DOM
`.css(property, value)`	Reads or directly sets an inline CSS style
`$("#id").datepicker()`	jQuery UI widget — converts a text input into an interactive calendar
`.dialog({...})`	jQuery UI widget — creates a draggable, modal popup window

Expected Behaviour:

```
Click "Hide Image"   -> Image fades out of view over 0.5s
Click "Add Student"  -> A new <li>New Student 3</li> appears at
the bottom of the list
Click "Highlight"    -> All list items get a yellow background
Click date input     -> A calendar pop-up appears for date
selection
Click "Open Dialog"  -> A modal box titled "Notice" appears
center-screen
```

Module 3J: Joomla — Introduction to CMS & Installation

1. Minute Rigorous Definition & Conceptual Core

- **CMS (Content Management System):** Software that allows non-programmers to create, edit, organise, and publish digital content through a graphical interface, without writing raw HTML/PHP for every page. The CMS handles the underlying database storage, templating, and rendering.

- **Joomla:** A free, open-source CMS written in PHP, using a MySQL/MariaDB database. It sits conceptually between simpler platforms like WordPress and heavier enterprise frameworks like Drupal — strong for structured, multi-user, multi-category websites (portals, membership sites, directories, e-commerce with extensions).
- **Joomla vs Other CMS:**

CMS	Best Suited For	Learning Curve
WordPress	Blogs, small business sites, simple content	Easiest
Joomla	Structured/complex sites, community portals, directories	Moderate
Drupal	Highly custom, enterprise-scale, developer-heavy sites	Steepest

- **Server Requirements:** A web server (Apache/Nginx), PHP (matching Joomla's supported version range), and a MySQL/MariaDB database — collectively often bundled locally as **XAMPP** or **WAMP** for development/practice.

2. Conceptual ASCII Diagram: Joomla Installation Flow

```
[ Download joomla_x.x.x.zip from joomla.org ]
                |
                v
[ Extract into htdocs/joomla (XAMPP) or public_html (live
server) ]
                |
                v
[ Create an empty MySQL database + a DB user via phpMyAdmin ]
                |
                v
[ Visit http://localhost/joomla in browser -> Joomla Web Installer
launches ]
                |
                v
   STEP 1: Site configuration (Site Name, Admin email, Admin
username/password)
                |
                v
   STEP 2: Database configuration (Host, DB name, DB user, DB
password, table prefix)
                |
```

```
                v
   STEP 3: Overview & pre-installation checks -> Click "Install"
                |
                v
[ Joomla writes configuration.php, populates DB tables ]
                |
                v
[ DELETE the /installation folder (Joomla refuses to run
otherwise, for security) ]
                |
                v
[ Site is live: Frontend at /  |  Backend admin at /
administrator ]
```

3. Step-by-Step Practical Walkthrough

```
1. Set up a local server environment (XAMPP/WAMP) and start Apache
+ MySQL.

2. Create a database:
   - Open phpMyAdmin (http://localhost/phpmyadmin)
   - Click "New" -> name it e.g. "joomla_lab_db" -> Collation:
utf8mb4_general_ci -> Create

3. Download & extract Joomla:
   - Get the latest .zip from https://www.joomla.org/download.html
   - Extract all contents into: C:/xampp/htdocs/joomla/

4. Run the Web Installer:
   - Navigate to: http://localhost/joomla
   - Fill in:
       Site Name:        "TU BCA Demo Site"
       Admin Email:      admin@example.com
       Admin Username:   admin (avoid the default "admin" on live
sites for security)
       Admin Password:   (strong password)

5. Database Configuration screen:
       Database Type:    MySQLi
       Host Name:        localhost
       Username:         root
       Password:         (blank on default XAMPP)
       Database Name:    joomla_lab_db
       Table Prefix:     random 4-letter prefix auto-generated
```

```
(e.g. "ab12_")

6. Finalise:
   - Review the overview screen (green checks = requirements met)
   - Click "Install"
   - Once complete, Joomla will PROMPT you to remove the /
installation folder
     -> Delete it manually via file manager/FTP, or click the
"Remove installation folder" button

7. Access points after installation:
       Frontend (public site):   http://localhost/joomla
       Backend (admin panel):    http://localhost/joomla/
administrator
```

4. Comprehensive Analysis Table

Requirement	Typical Value
PHP Version	Matches the Joomla release's documented compatibility range
Database	MySQL or MariaDB
Web Server	Apache or Nginx
Table Prefix	Random string recommended (security — harder to guess table names for SQL injection attempts)
`/installation` folder	**Must** be deleted/renamed after setup, or Joomla blocks all access as a safeguard
`configuration.php`	Auto-generated file storing DB credentials and site settings — the Joomla equivalent of WordPress's `wp-config.php`

Module 3K: Joomla — Backend, Customization & Extensions

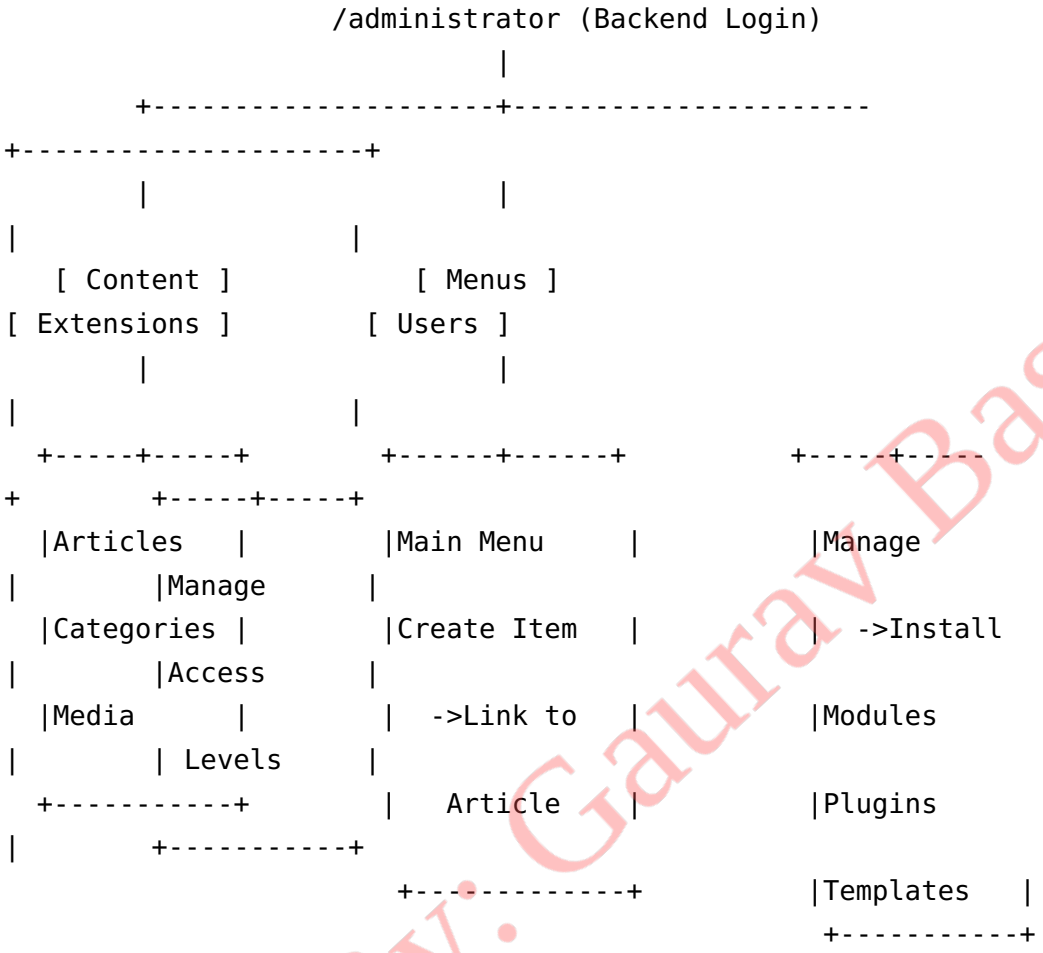
1. Minute Rigorous Definition & Conceptual Core

- **Joomla Backend (`/administrator`):** The password-protected control panel where all site management happens — content, users, menus, extensions, and global configuration.
- **Handling the Backend:** Key panels include the **Control Panel dashboard**, **Content → Articles** (create/edit content), **Content → Categories** (organise articles), **Menus** (build navigation linking to content), **Users** (manage accounts and access levels), and **System → Global Configuration** (site name, SEO settings, error reporting, caching).
- **Customization:** Achieved primarily through **Templates** (control visual design/layout) and **Modules** (small content blocks like login forms, menus, or "latest news" placed in specific template positions) combined with the **Template Manager**'s style/position assignment.
- **Extensions:** Joomla's plugin ecosystem, installed via **Extensions → Manage → Install**. Four main extension types exist:

Extension Type	Purpose	Example
Component	A full mini-application; the largest extension type, has its own admin section	A shopping cart, a forum, a directory listing
Module	A small, positionable content block displayed around the main content	Login form, breadcrumbs, latest news box
Plugin	Event-driven code that runs automatically in response to system events	Content filtering, SEF URL rewriting, spam filtering
Template	Controls the overall look, layout, and positions available for modules	A responsive front-end theme

- **Installing an Extension:** Upload a `.zip` package via **Extensions → Manage → Install**, or install directly from the **Joomla Extensions Directory (JED)** URL/web install tab.

2. Conceptual ASCII Diagram: Joomla Backend Navigation Map



3. Production Walkthrough & Customization Steps

A. Creating Content & Navigation

1. Content -> Articles -> New: write title + body -> assign a Category -> Save & Close
2. Menus -> Main Menu -> New Menu Item -> Menu Item Type: "Single Article"
   -> Select the article created above -> Save & Close
3. Visit the frontend: the new menu link now appears and opens the article.

B. Customizing Appearance via Templates

1. Extensions -> Templates -> Styles -> select the active template (e.g. "Cassiopeia")
2. Adjust: Site Name display, colours, logo upload, layout options
3. Extensions -> Templates -> Styles -> "Set as Default" to make it site-wide

```
C. Positioning Modules (small content blocks)
   1. Content -> Site Modules -> New -> choose module type (e.g.
"Latest News")
   2. Set: Title, Position (e.g. "sidebar-right"), Menu Assignment
(which pages show it), Status: Published
   3. Save -> module now appears in the chosen template position
on selected pages

D. Installing a New Extension
   1. Download a component/module .zip (e.g. a contact-form
component) from JED or a vendor site
   2. Extensions -> Manage -> Install -> "Upload Package File" ->
select the .zip -> Install
   3. The new extension appears under its relevant menu
(Components / Modules / Plugins)
   4. Configure and Publish it like any built-in extension

E. Global Configuration Essentials
   - System -> Global Configuration -> Site tab: Site Name,
Default editor, Site Offline toggle
   - System -> Global Configuration -> System tab: caching,
session lifetime
   - System -> Global Configuration -> Server tab: SEF (Search
Engine Friendly) URLs, error reporting
```

4. Comprehensive Analysis Table

Backend Area	What It Controls
Content → Articles	The actual written content (pages/posts equivalent)
Content → Categories	Grouping/organisation of articles
Menus	The clickable navigation structure connecting URLs to content
Site Modules	Small reusable content blocks placed in template positions
Extensions → Manage	Installing/uninstalling Components, Modules, Plugins, Templates
Users → Access Levels	Role-based permissions (Public, Registered, Author, Editor, Publisher, Administrator, Super User)

Backend Area	What It Controls
Global Configuration	Site-wide behaviour: SEO, caching, error display, session handling

Module 3L: Joomla — Template & Module/ Component Development, Artisteer, and MVC

1. Minute Rigorous Definition & Conceptual Core

- **Template Development:** A Joomla template is a folder of PHP, CSS, and XML files defining the site's structural layout and design. The core file `templateDetails.xml` declares the template's name, author, and available **module positions** (`<position>sidebar-right</position>`); `index.php` uses `<jdoc:include type="modules" name="sidebar-right" />` tags to render whatever modules are assigned to each named position.
- **Artisteer:** A third-party, visual **What-You-See-Is-What-You-Get (WYSIWYG) design tool/IDE** historically used to generate Joomla (and WordPress) templates by dragging and dropping design elements, without hand-writing CSS — it exports a ready-to-install template package.
- **Module Development:** A custom Joomla module is a small extension package (folder + XML manifest + PHP entry file) that outputs a self-contained piece of content, registered so it can be placed in any template position via the backend, just like a built-in module.
- **Component Development:** A custom Joomla **component** is the most substantial extension type, following Joomla's **MVC (Model-View-Controller)** architecture, and typically ships with **both** a frontend (`site`) part and a backend (`admin`) management part, plus its own database tables.
- **MVC (Model-View-Controller) Pattern:**
  - **Model:** Handles data — retrieving from and saving to the database. Contains no display logic.
  - **View:** Prepares and renders the presentation/output (HTML) using data handed to it by the Controller.
  - **Controller:** Receives the incoming request, decides what action to take, asks the Model for data, and hands that data to the

correct View. It is the coordinator, containing minimal logic itself.

2. Conceptual ASCII Diagram: Joomla MVC Request Flow

```
   Browser Request: index.php?option=com_students&view=list
                                |
                                v
                    +-------------------+
                    |   CONTROLLER      |   <-- Reads the
request, decides what to do
                    +-------------------+
                        |           |
                asks for data   selects a View
                        |           |
                        v           v
                 +-----------+  +-----------+
                 |   MODEL   |  |   VIEW    |
                 | queries DB |  | formats   |
                 | returns   |->| the Model's|
                 | raw data  |  | data as   |
                 +-----------+  | HTML      |
                                +-----------+
                                      |
                                      v
                                [ Final HTML sent to Browser ]

TEMPLATE MODULE-POSITION RENDERING:
   templateDetails.xml declares positions: "sidebar-right",
"footer", "top"
                                |
   index.php:  <jdoc:include type="modules" name="sidebar-right" /
>
                                |
   Backend assigns "Latest News" module -> position "sidebar-
right"
                                |
   Result: "Latest News" module HTML is injected exactly there
```

3. Production-Ready Structural Examples

Minimal Template Structure (`templates/my_template/`):

```
my_template/
├── templateDetails.xml      <- Manifest: name, positions, files
list
├── index.php                <- Main layout file, uses
<jdoc:include> tags
├── css/
│   └── template.css
└── images/
    └── logo.png
```

```xml
<!-- templateDetails.xml (simplified) -->
<extension type="template" client="site" method="upgrade">
    <name>my_template</name>
    <version>1.0.0</version>
    <positions>
        <position>top</position>
        <position>sidebar-right</position>
        <position>footer</position>
    </positions>
    <files>
        <filename>index.php</filename>
    </files>
</extension>
```

```php
<!-- index.php (simplified layout) -->
<!DOCTYPE html>
<html>
<head>
    <jdoc:include type="head" />
    <link rel="stylesheet" href="<?php echo $this->baseurl; ?>/
        templates/my_template/css/template.css">
</head>
<body>
    <header><jdoc:include type="modules" name="top" /></header>
    <main><jdoc:include type="component" /></main>   <!-- Main
        article/component output -->
    <aside><jdoc:include type="modules" name="sidebar-right" /></
        aside>
    <footer><jdoc:include type="modules" name="footer" /></footer>
</body>
</html>
```

Minimal Custom Module Structure (`modules/mod_greeting/`):

```
mod_greeting/
├── mod_greeting.xml     <- Manifest
```

```
├── mod_greeting.php     <- Entry point, calls the helper
├── helper.php           <- Business logic
└── tmpl/
    └── default.php      <- Presentation (View) template
```

```php
// mod_greeting.php
<?php
defined('_JEXEC') or die;   // Prevent direct access outside
        Joomla
require_once __DIR__ . '/helper.php';

$greeting = ModGreetingHelper::getGreeting();
require JModuleHelper::getLayoutPath('mod_greeting');   // Loads
        tmpl/default.php

// tmpl/default.php
<div class="greeting-module">
    <p><?php echo htmlspecialchars($greeting); ?></p>
</div>
```

Minimal Component MVC Skeleton (`components/com_students/`):

```
com_students/
├── site/
│   ├── controller.php          <- CONTROLLER
│   ├── models/
│   │   └── students.php        <- MODEL
│   └── views/
│       └── list/
│           ├── view.html.php   <- VIEW
│           └── tmpl/default.php
└── admin/
    └── (backend management screens)
```

```php
// site/models/students.php  (MODEL — data access only)
<?php
defined('_JEXEC') or die;

class StudentsModelStudents extends JModelList {
    public function getStudents() {
        $db = JFactory::getDbo();
        $query = $db->getQuery(true)->select('*')-
        >from('#__students');
        $db->setQuery($query);
        return $db->loadObjectList();   // Returns data ONLY, no
        HTML here
```

```
    }
}

// site/views/list/view.html.php  (VIEW — presentation only)
<?php
defined('_JEXEC') or die;

class StudentsViewList extends JViewLegacy {
    public function display($tpl = null) {
        $this->students =
        $this->get('Students');   // Pulled from the Model
        parent::display($tpl);                          // Renders
        tmpl/default.php
    }
}
```

4. Comprehensive Analysis Table

Layer	Responsibility	Should NOT Contain
Model	Database queries, business/data logic	HTML, presentation code
View	Formatting retrieved data into HTML	Direct SQL queries
Controller	Routing the request to the right Model + View	Complex business logic
Template	Overall page skeleton + module positions	Component-specific business logic
Module	Small, positionable, reusable content block	Full page layout
Component	Full mini-application, own DB tables, uses MVC	—
Artisteer	Visual drag-and-drop generator for template packages	Hand-written PHP business logic

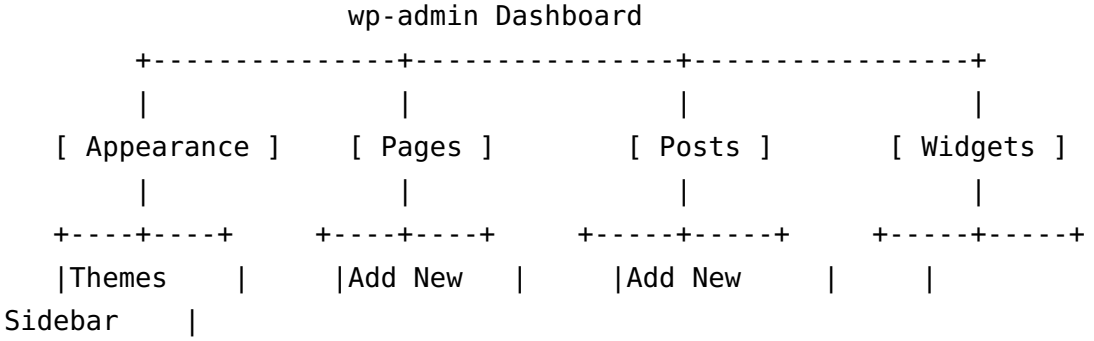
Module 3M: WordPress Administrator Level

1. Minute Rigorous Definition & Conceptual Core

WordPress is the world's most widely used CMS, prized for its gentle learning curve, enormous theme/plugin ecosystem, and its clear separation between **Pages** (static, standalone content — "About Us", "Contact") and **Posts** (time-stamped, chronologically organised content — a blog feed, typically tagged and categorised).

- **Theme Integration:** A WordPress **Theme** is a package of PHP template files, CSS, and assets controlling the entire site's visual presentation. Installed via **Appearance → Themes → Add New**, then **activated**. The **Customizer** (**Appearance → Customize**) allows live-preview edits (logo, colours, menus, widgets) without touching code. Many modern themes also support the **Full Site Editor / block-based themes** for direct visual layout editing.
- **Creating Pages: Pages → Add New** — used for static, hierarchical content that doesn't belong in the chronological blog stream (Home, About, Contact, Services). Pages can have **Parent/Child** relationships (e.g., "Services" as parent, "Web Design" as a child page) but do **not** use categories/tags.
- **Managing Posts: Posts → Add New** — used for blog-style, dated content. Posts are organised using **Categories** (broad grouping, hierarchical) and **Tags** (specific keywords, non-hierarchical), and appear in the site's chronological blog feed and RSS.
- **Managing Widgets: Widgets** are small content/functionality blocks (search bar, recent posts, calendar, custom text/HTML) placed into **theme-defined widget areas** (commonly sidebar, footer). Managed under **Appearance → Widgets**, using the block-based Widgets screen.

2. Conceptual ASCII Diagram: WordPress Admin Structure



```
   | ->Activate|     |All Pages |     |Categories  |     |
Footer     |
   |Customize |     |Parent/   |     |Tags        |     |
(theme-    |
   |Widgets   |     | Child    |     |All Posts   |     |
defined   |
   +----------+     +---------+     +-----------+     | areas)
|

+-----------+

  PAGES vs POSTS:
  Page: "About Us"  -- static, no date, no category, can have a
parent page
  Post: "10 Tips for New BCA Students" -- dated, categorized
"Education", tagged "tips, BCA"
```

3. Step-by-Step Practical Walkthrough

```
A. Theme Integration
   1. Appearance -> Themes -> Add New -> search or upload a
theme .zip -> Install -> Activate
   2. Appearance -> Customize -> adjust Site Identity (logo,
title, tagline),
      Colors, Menus, Homepage Settings -> Publish

B. Creating a Page
   1. Pages -> Add New
   2. Enter Title: "About Us"; write content in the Block Editor
   3. (Optional) Page Attributes panel -> set a Parent page for
hierarchy
   4. Set Featured Image if needed -> Publish

C. Managing Posts
   1. Posts -> Categories -> Add New Category: "Announcements"
   2. Posts -> Add New -> Title: "Semester Exam Routine Released"
   3. Assign Category: "Announcements"; add Tags: "exam, routine,
BCA"
   4. Set Featured Image -> Publish
   5. Posts -> All Posts: edit, trash, or quick-edit any existing
post from this list

D. Managing Widgets
   1. Appearance -> Widgets
```

```
   2. Select a widget area (e.g. "Sidebar")
   3. Add a widget block: e.g. "Recent Posts", "Search", or
"Custom HTML"
   4. Configure its settings (title, number of items to show) ->
Update/Save
   5. Widget now appears automatically on every page/post using
that sidebar area
```

4. Comprehensive Analysis Table

Feature	Pages	Posts
Chronological?	No	Yes (dated, reverse-chronological feed)
Organisation	Parent/Child hierarchy	Categories (hierarchical) + Tags (flat)
Typical Use	About, Contact, Services	Blog articles, news, announcements
Appears in RSS Feed	No	Yes

Concept	Location in wp-admin	Purpose
Theme Activation	Appearance → Themes	Changes entire site design
Customizer	Appearance → Customize	Live-preview visual tweaks
New Page	Pages → Add New	Static standalone content
New Post	Posts → Add New	Dated, categorised blog content
Widgets	Appearance → Widgets	Small content blocks in sidebar/ footer areas

End of Unit 3 — Advanced Server Side Scripting